

Tugas Analisis Multimedia: Image (Citra Digital)

Mata Kuliah: Pengolahan Citra
Nama: Didi Tri Saputra NIM: 122400048

Link Repository GitHub: [Pengolahan_Multimedia](#) **Link Percakapan AI:**

<https://claude.ai/share/3e545975-ee52-405f-b0d5-c1ca2e4887a8>

Deskripsi Tugas

Tugas ini bertujuan untuk memahami **representasi dasar data citra digital (image)** melalui praktik langsung memuat data, visualisasi komponen warna, serta melakukan analisis spasial sederhana menggunakan berbagai teknik dasar pengolahan citra.

Anda akan bekerja dengan satu atau beberapa gambar (foto diri, objek, atau lingkungan sekitar) untuk:

- Mengamati struktur data piksel dan channel warna (RGB, Grayscale, HSV, dsb.)
- Menganalisis perbedaan hasil visualisasi antar representasi warna
- Melakukan eksplorasi sederhana terhadap transformasi citra (cropping, filtering, edge detection, dll.)
- Menyimpulkan pengaruh setiap tahap pemrosesan terhadap persepsi visual

Fokus tugas ini adalah pada **pemahaman konsep representasi spasial citra digital** dan **interpretasi hasil visualisasi**, bukan pada manipulasi kompleks atau penerapan model pembelajaran mesin.

Soal 1 — Cropping dan Konversi Warna

- Ambil sebuah gambar diri Anda (*selfie*) menggunakan kamera atau smartphone.
- Lakukan **cropping secara manual** untuk menghasilkan dua potongan:
 - Cropping **kotak persegi pada area wajah**.
 - Cropping **persegi panjang pada area latar belakang**.
- Resize hasil crop menjadi **920×920 piksel**.
- Konversi gambar menjadi **grayscale** dan **HSV**, lalu tampilkan ketiganya berdampingan.
- Tambahkan **anotasi teks** berisi nama Anda di atas kepala pada gambar hasil crop.

- Gaya teks (font, warna, posisi, ukuran, ketebalan) **dibebaskan**.
- Jelaskan efek **cropping** dan **perubahan warna** menggunakan **Markdown**.

```
In [6]: """
=====
    Soal 1 – Cropping dan Konversi Warna
    Pengolahan Citra Digital
=====
"""

import numpy as np
import matplotlib.pyplot as plt
import cv2
import os

NAMA_ANDA = "Didi Tri Saputra"
PATH_FOTO = PATH_FOTO = r"C:\Users\HP\Pengolahan_Multimedia\data\selfie untuk citr

# Koordinat crop dalam format (y1, y2, x1, x2)
CROP_WAJAH = (1050, 2167, 307, 1424)
CROP_BG = (0, 700, 0, 1793)

print("=" * 60)
print(" SOAL 1 – CROPPING DAN KONVERSI WARNA")
print("=" * 60)

print("\n[1] Memuat gambar...")

img = cv2.imread(PATH_FOTO)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

print(f"    Ukuran gambar asli: {img_rgb.shape[1]}x{img_rgb.shape[0]} px")

plt.figure(figsize=(5, 7))
plt.imshow(img_rgb)
plt.title(f'Gambar Asli – {img_rgb.shape[1]}x{img_rgb.shape[0]} px', fontsize=12)
plt.axis('off')
plt.tight_layout()
plt.show()

print("\n[2] Menampilkan preview area crop...")

y1_w, y2_w, x1_w, x2_w = CROP_WAJAH
y1_b, y2_b, x1_b, x2_b = CROP_BG

img_preview = img_rgb.copy()

cv2.rectangle(img_preview, (x1_w, y1_w), (x2_w, y2_w), (0, 255, 0), 10)
cv2.rectangle(img_preview, (x1_b, y1_b), (x2_b, y2_b), (0, 200, 255), 10)
cv2.putText(img_preview, 'Wajah (Square)', (x1_w + 10, y1_w + 80),
            cv2.FONT_HERSHEY_SIMPLEX, 2.5, (0, 255, 0), 6)
cv2.putText(img_preview, 'Background (Rect)', (x1_b + 10, y1_b + 120),
            cv2.FONT_HERSHEY_SIMPLEX, 2.5, (0, 200, 255), 6)

# Tampilkan preview skala kecil agar muat di layar
preview_kecil = cv2.resize(img_preview, (450, int(450 * img_preview.shape[0] / img_
```

```

plt.figure(figsize=(5, 7))
plt.imshow(preview_kecil)
plt.title('Preview Area Crop\n(Hijau = Wajah | Cyan = Background)', fontsize=12)
plt.axis('off')
plt.tight_layout()
plt.show()

print("\n[3] Melakukan cropping manual...")

# Crop menggunakan slicing numpy
crop_wajah = img_rgb[y1_w:y2_w, x1_w:x2_w]
crop_bg     = img_rgb[y1_b:y2_b, x1_b:x2_b]

print(f"    Crop wajah      : {crop_wajah.shape[1]}x{crop_wajah.shape[0]} px")
print(f"    Crop background : {crop_bg.shape[1]}x{crop_bg.shape[0]} px")

fig, axs = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle('Hasil Cropping (Sebelum Resize)', fontsize=13, fontweight='bold')

axs[0].imshow(crop_wajah)
axs[0].set_title(f'Crop Wajah (Kotak Persegi)\n{crop_wajah.shape[1]}x{crop_wajah.sh
axs[0].axis('off')

axs[1].imshow(crop_bg)
axs[1].set_title(f'Crop Background (Persegi Panjang)\n{crop_bg.shape[1]}x{crop_bg.s
axs[1].axis('off')

plt.tight_layout()
plt.show()

print("\n[4] Resize ke 920x920 piksel...")

wajah_920 = cv2.resize(crop_wajah, (920, 920), interpolation=cv2.INTER_LANCZOS4)
bg_920    = cv2.resize(crop_bg, (920, 920), interpolation=cv2.INTER_LANCZOS4)

print(f"    Shape setelah resize: {wajah_920.shape}")

print(f"\n[5] Menambahkan anotasi nama: '{NAMA_ANDA}'...")

def tambah_nama(img, nama):
    out = img.copy()
    font = cv2.FONT_HERSHEY_DUPLEX
    scale = 1.4
    tebal = 2
    ts = cv2.getTextSize(nama, font, scale, tebal)[0]
    x = (out.shape[1] - ts[0]) // 2
    y = 55
    cv2.putText(out, nama, (x + 2, y + 2), font, scale, (0, 0, 0), tebal + 3, cv2.L
    cv2.putText(out, nama, (x, y), font, scale, (255, 255, 0), tebal, cv2.LINE_AA)
    return out

wajah_annotated = tambah_nama(wajah_920, NAMA_ANDA)
bg_annotated    = tambah_nama(bg_920, NAMA_ANDA)

```

```

fig, axs = plt.subplots(1, 2, figsize=(12, 6))
fig.suptitle('Hasil Crop 920x920 + Anotasi Nama', fontsize=13, fontweight='bold')

axs[0].imshow(wajah_annotated)
axs[0].set_title('Crop Wajah (920x920 px)', fontsize=11)
axs[0].axis('off')

axs[1].imshow(bg_annotated)
axs[1].set_title('Crop Background (920x920 px)', fontsize=11)
axs[1].axis('off')

plt.tight_layout()
plt.show()

# Simpan hasil crop
cv2.imwrite('crop_wajah_920.png', cv2.cvtColor(wajah_annotated, cv2.COLOR_RGB2GRAY))
cv2.imwrite('crop_background_920.png', cv2.cvtColor(bg_annotated, cv2.COLOR_RGB2GRAY))
print("    ✓ Tersimpan: crop_wajah_920.png")
print("    ✓ Tersimpan: crop_background_920.png")

print("\n[6] Konversi warna (RGB → Grayscale & HSV)...")

def tampilkan_konversi(img_rgb_input, judul, nama_file):
    img_gray = cv2.cvtColor(img_rgb_input, cv2.COLOR_RGB2GRAY)
    img_hsv = cv2.cvtColor(img_rgb_input, cv2.COLOR_RGB2HSV)

    fig, axs = plt.subplots(1, 3, figsize=(15, 5))
    fig.suptitle(f'Konversi Warna – {judul}', fontsize=13, fontweight='bold')

    axs[0].imshow(img_rgb_input)
    axs[0].set_title('Original (RGB)', fontsize=11)
    axs[0].axis('off')

    axs[1].imshow(img_gray, cmap='gray')
    axs[1].set_title('Grayscale', fontsize=11)
    axs[1].axis('off')

    axs[2].imshow(img_hsv)
    axs[2].set_title('HSV', fontsize=11)
    axs[2].axis('off')

    plt.tight_layout()
    plt.savefig(nama_file, dpi=150, bbox_inches='tight')
    plt.show()
    print(f"    ✓ Tersimpan: {nama_file}")

tampilkan_konversi(wajah_annotated, 'Crop Wajah', 'konversi_wajah.png')
tampilkan_konversi(bg_annotated, 'Crop Background', 'konversi_background.png')

print("\n[7] Memisahkan channel warna crop wajah...")

red_ch = wajah_annotated[:, :, 0]
green_ch = wajah_annotated[:, :, 1]
blue_ch = wajah_annotated[:, :, 2]

```



```

fig, axs = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Pemisahan Channel Warna – Crop Wajah', fontsize=13, fontweight='bold')

axs[0].imshow(red_ch, cmap='Reds')
axs[0].set_title('Channel Merah (R)', fontsize=11)
axs[0].axis('off')

axs[1].imshow(green_ch, cmap='Greens')
axs[1].set_title('Channel Hijau (G)', fontsize=11)
axs[1].axis('off')

axs[2].imshow(blue_ch, cmap='Blues')
axs[2].set_title('Channel Biru (B)', fontsize=11)
axs[2].axis('off')

plt.tight_layout()
plt.show()

print("\n" + "=" * 60)
print("  SELESAI! File output yang dihasilkan:")
print("=" * 60)
print("  ✓ crop_wajah_920.png")
print("  ✓ crop_background_920.png")
print("  ✓ konversi_wajah.png")
print("  ✓ konversi_background.png")
print("=" * 60)

```

```

=====
SOAL 1 – CROPPING DAN KONVERSI WARNA
=====

```

```

[1] Memuat gambar...
    Ukuran gambar asli: 1793x2391 px

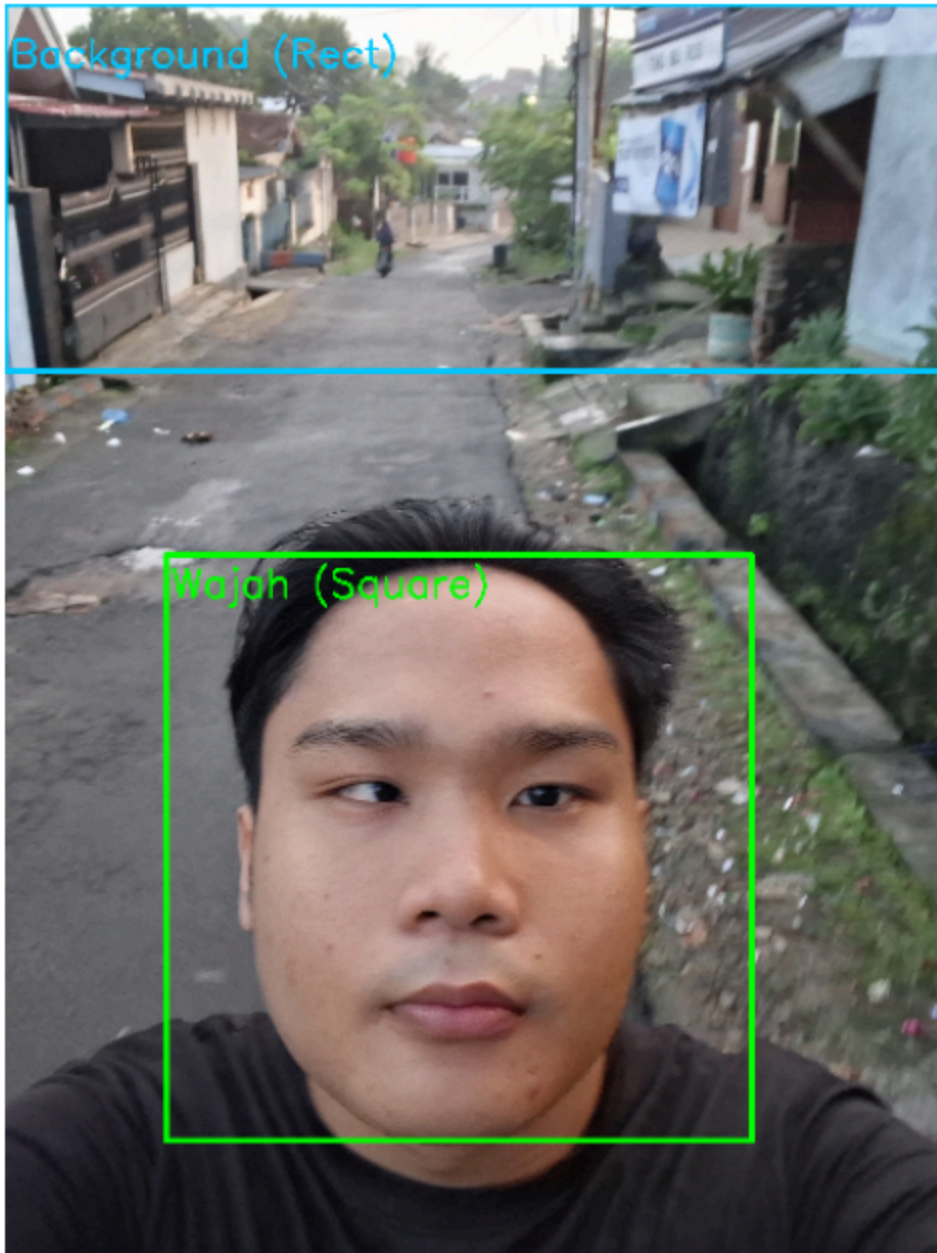
```

Gambar Asli — 1793x2391 px



[2] Menampilkan preview area crop...

Preview Area Crop
(Hijau = Wajah | Cyan = Background)



[3] Melakukan cropping manual...
Crop wajah : 1117x1117 px
Crop background : 1793x700 px

Hasil Cropping (Sebelum Resize)

Crop Wajah (Kotak Persegi)
1117x1117 px



Crop Background (Persegi Panjang)
1793x700 px



[4] Resize ke 920x920 piksel...

Shape setelah resize: (920, 920, 3)

[5] Menambahkan anotasi nama: 'Didi Tri Saputra'...

Hasil Crop 920x920 + Anotasi Nama

Crop Wajah (920x920 px)



Crop Background (920x920 px)



✓ Tersimpan: crop_wajah_920.png

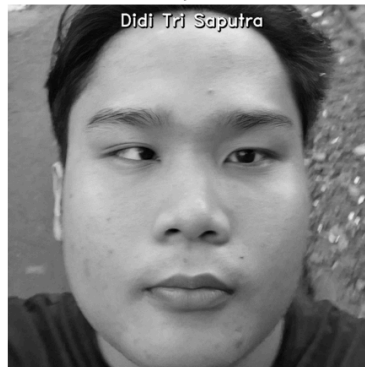
✓ Tersimpan: crop_background_920.png

[6] Konversi warna (RGB → Grayscale & HSV)...

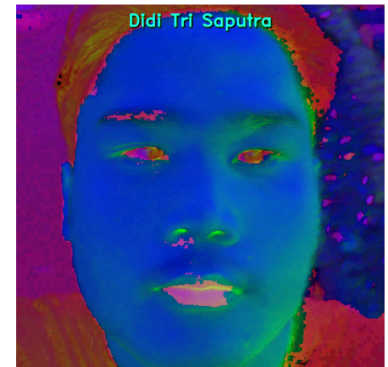
Original (RGB)



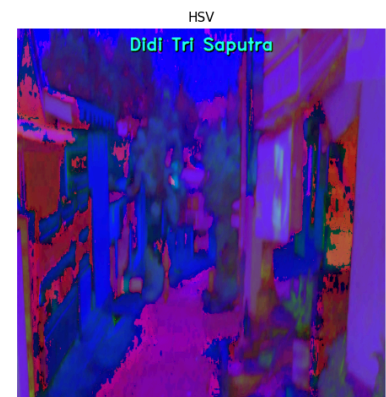
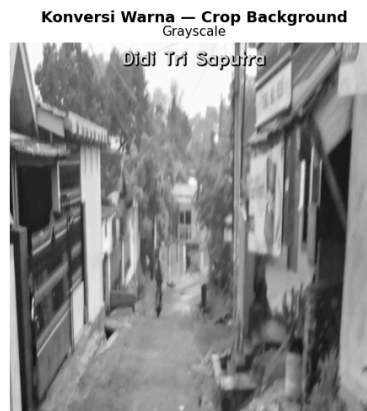
Konversi Warna — Crop Wajah
Grayscale



HSV

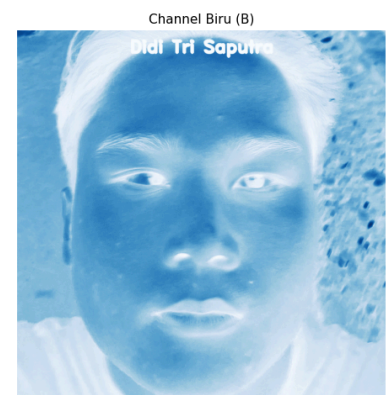
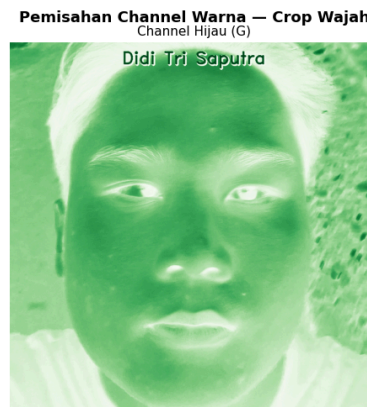


✓ Tersimpan: konversi_wajah.png



✓ Tersimpan: konversi_background.png

[7] Memisahkan channel warna crop wajah...



=====

SELESAI! File output yang dihasilkan:

=====

- ✓ crop_wajah_920.png
 - ✓ crop_background_920.png
 - ✓ konversi_wajah.png
 - ✓ konversi_background.png
- =====

Penjelasan — Soal 1: Cropping dan Konversi Warna

Nama: Didi Tri Saputra

Mata Kuliah: Pengolahan Citra Digital

1. Proses Cropping

Apa itu Cropping?

Cropping adalah proses memotong sebagian area dari sebuah gambar berdasarkan koordinat tertentu, sehingga menghasilkan gambar baru yang hanya berisi Region of

Interest (ROI) yang diinginkan. Pada Python dengan NumPy, cropping dilakukan menggunakan slicing array: `img[y1:y2, x1:x2]`.

Dua Jenis Crop yang Digunakan

Jenis Crop	Bentuk	Area	Ukuran Setelah Resize
Crop Wajah	Kotak Persegi (Square)	Area wajah	920×920 px
Crop Background	Persegi Panjang (Rectangle)	Area latar belakang	920×920 px

Efek Cropping

Aspek	Penjelasan
Fokus	Menghilangkan bagian yang tidak relevan sehingga subjek utama lebih menonjol
Ukuran file	Berkurang karena jumlah piksel lebih sedikit dari gambar asli
Komposisi	Crop kotak (1:1) memberikan kesan simetris dan terfokus pada subjek
Kehilangan konteks	Bagian di luar area crop tidak dapat dipulihkan tanpa gambar asli

2. Konversi Ruang Warna

2.1 Original (RGB)

Model warna standar yang merepresentasikan setiap piksel dengan tiga kanal: **Red, Green, Blue**. Nilai tiap kanal berkisar antara 0 hingga 255. Model ini paling umum digunakan untuk menampilkan gambar di layar monitor.

2.2 Grayscale

Setiap piksel dikonversi menjadi satu nilai intensitas (0 = hitam, 255 = putih) menggunakan rumus pembobotan perceptual:

Efek Grayscale:

- Menghilangkan informasi warna (hue & saturation)
- Menyisakan hanya informasi kecerahan (luminance)
- Digunakan untuk deteksi tepi, analisis tekstur, dan OCR

2.3 HSV (Hue, Saturation, Value)

Representasi warna yang lebih sesuai dengan cara manusia mempersepsikan warna.

Kanal	Rentang (OpenCV)	Makna
H (Hue)	0–179	Jenis warna (merah=0, hijau=60, biru=120)
S (Saturation)	0–255	Kepekatan warna (0=abu-abu, 255=penuh)
V (Value)	0–255	Kecerahan (0=hitam, 255=sangat terang)

Efek & Keunggulan HSV:

- Memisahkan informasi warna (H) dari kecerahan (V)
- Lebih tahan terhadap perubahan pencahayaan dibanding RGB
- Sangat berguna untuk segmentasi warna, misalnya deteksi warna kulit atau objek berwarna tertentu

3. Perbandingan Ruang Warna

	Original (RGB)	Grayscale	HSV
Jumlah Kanal	3 (R, G, B)	1 (intensitas)	3 (H, S, V)
Informasi Warna	✓ Lengkap	✗ Tidak ada	✓ Terpisah di kanal H
Informasi Kecerahan	Tersebar di R+G+B	✓ Eksplisit	✓ Kanal V
Kegunaan Utama	Tampilan umum	Deteksi tepi & tekstur	Segmentasi berbasis warna
Ketahanan Cahaya	Rendah	Rendah	Tinggi (H tidak berubah)

4. Kesimpulan

- **Cropping** membantu mengisolasi ROI yang relevan dan mengurangi noise visual dari area yang tidak dibutuhkan dalam analisis.
- **Grayscale** menyederhanakan data dengan mempertahankan informasi struktural (bentuk dan tepi) tanpa informasi warna.
- **HSV** memisahkan warna dari kecerahan sehingga lebih efektif untuk segmentasi dan deteksi objek dalam kondisi pencahayaan yang bervariasi.

Dibuat menggunakan Python · OpenCV · NumPy · Matplotlib

Soal 2 — Manipulasi Channel Warna RGB

- Gunakan gambar hasil crop dari Soal 1.
- Konversikan gambar ke ruang warna **RGB**.

- Lakukan manipulasi channel warna dengan cara:
 - **Naikkan intensitas channel merah sebanyak 50 poin** (maksimum 255).
 - **Turunkan intensitas channel biru sebanyak 30 poin** (minimum 0).
- Teknik atau cara menaikkan/menurunkan intensitas **dibebaskan**, asalkan logis dan hasilnya terlihat.
- Gabungkan kembali channel warna dan **simpan gambar hasil modifikasi dalam format .png**.
- **Tampilkan histogram per channel (R, G, B)** untuk gambar asli dan hasil modifikasi menggunakan `matplotlib.pyplot.hist`.
- Jelaskan dampak perubahan RGB pada warna gambar dalam sel **Markdown**.

```
In [7]: """
=====
    Soal 2 – Manipulasi Channel Warna RGB
    Pengolahan Citra Digital
=====
"""

import numpy as np
import matplotlib.pyplot as plt
import cv2

print("=" * 60)
print("  SOAL 2 – MANIPULASI CHANNEL WARNA RGB")
print("=" * 60)
print("\n[1] Menggunakan gambar hasil crop Soal 1...")

img_rgb = wajah_annotated.copy()

print(f"    Shape  : {img_rgb.shape}")
print(f"    Dtype  : {img_rgb.dtype}")
print(f"    Format : RGB (dari Soal 1)")

plt.figure(figsize=(5, 5))
plt.imshow(img_rgb)
plt.title('Gambar Input – Crop Wajah (RGB)', fontsize=12)
plt.axis('off')
plt.tight_layout()
plt.show()

print("\n[2] Memisahkan channel R, G, B...")

R = img_rgb[:, :, 0].copy()
G = img_rgb[:, :, 1].copy()
B = img_rgb[:, :, 2].copy()

print(f"    R – mean: {R.mean():.1f} | G – mean: {G.mean():.1f} | B – mean: {B.mean():.1f}")

fig, axs = plt.subplots(1, 3, figsize=(15, 5))
```



```

fig.suptitle('Channel Warna Asli (Sebelum Manipulasi)', fontsize=13, fontweight='bold')
axs[0].imshow(R, cmap='Reds'); axs[0].set_title(f'Channel Merah (R)\nmean={R.mean()}')
axs[1].imshow(G, cmap='Greens'); axs[1].set_title(f'Channel Hijau (G)\nmean={G.mean()}')
axs[2].imshow(B, cmap='Blues'); axs[2].set_title(f'Channel Biru (B)\nmean={B.mean()}')
plt.tight_layout()
plt.show()

print("\n[3] Memanipulasi channel (R+50, B-30)...")

R_mod = np.clip(R.astype(np.int32) + 50, 0, 255).astype(np.uint8)
G_mod = G.copy()
B_mod = np.clip(B.astype(np.int32) - 30, 0, 255).astype(np.uint8)

print(f"    R: {R.mean():.1f} → {R_mod.mean():.1f} (+50)")
print(f"    G: {G.mean():.1f} → {G_mod.mean():.1f} (tidak berubah)")
print(f"    B: {B.mean():.1f} → {B_mod.mean():.1f} (-30)")

print("\n[4] Menggabungkan channel dan menyimpan...")

img_modif = np.stack([R_mod, G_mod, B_mod], axis=2)

fig, axs = plt.subplots(1, 2, figsize=(12, 6))
fig.suptitle('Perbandingan Sebelum & Sesudah Manipulasi RGB', fontsize=13, fontweight='bold')
axs[0].imshow(img_rgb)
axs[0].set_title('Gambar Asli (Original)', fontsize=11)
axs[0].axis('off')
axs[1].imshow(img_modif)
axs[1].set_title('Hasil Modifikasi (R+50, B-30)', fontsize=11)
axs[1].axis('off')
plt.tight_layout()
plt.show()
cv2.imwrite('hasil_modifikasi_rgb.png', cv2.cvtColor(img_modif, cv2.COLOR_RGB2BGR))

import os
print(f"    ✓ Tersimpan di: {os.path.join(os.getcwd(), 'hasil_modifikasi_rgb.png')}")

print("\n[5] Menampilkan histogram per channel...")

channels = ['Red', 'Green', 'Blue']
colors = ['red', 'green', 'blue']
asli_chs = [R, G, B]
modif_chs = [R_mod, G_mod, B_mod]
labels = ['(R+50)', '(tidak berubah)', '(B-30)']

fig, axs = plt.subplots(2, 3, figsize=(15, 8))
fig.suptitle('Histogram Channel Warna – Asli vs Modifikasi', fontsize=13, fontweight='bold')

for i, (ch, color, ch_a, ch_m, lbl) in enumerate(zip(channels, colors, asli_chs, modif_chs, labels)):
    axs[0, i].hist(ch_a.ravel(), bins=256, range=(0, 256), color=color, alpha=0.8)
    axs[0, i].set_title(f'Asli – Channel {ch}', fontsize=11)
    axs[0, i].set_xlabel('Intensitas (0-255)')
    axs[0, i].set_ylabel('Jumlah Pixel')
    axs[0, i].set_xlim([0, 256])

    axs[1, i].hist(ch_m.ravel(), bins=256, range=(0, 256), color=color, alpha=0.8)
    axs[1, i].set_title(f'Modif – Channel {ch} {lbl}', fontsize=11)

```

```

    axs[1, i].set_xlabel('Intensitas (0-255)')
    axs[1, i].set_ylabel('Jumlah Piksel')
    axs[1, i].set_xlim([0, 256])

plt.tight_layout()
plt.savefig('histogram_rgb.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"    ✓ Tersimpan di: {os.path.join(os.getcwd(), 'histogram_rgb.png')}")

print("\n" + "=" * 60)
print("    SELESAI! File output:")
print("=" * 60)
print(f"    ✓ {os.path.join(os.getcwd(), 'hasil_modifikasi_rgb.png')}")
print(f"    ✓ {os.path.join(os.getcwd(), 'histogram_rgb.png')}")
print("=" * 60)

```

=====

SOAL 2 – MANIPULASI CHANNEL WARNA RGB

=====

[1] Menggunakan gambar hasil crop Soal 1...

Shape : (920, 920, 3)

Dtype : uint8

Format : RGB (dari Soal 1)

Gambar Input — Crop Wajah (RGB)



[2] Memisahkan channel R, G, B...

R – mean: 129.3 | G – mean: 107.0 | B – mean: 99.6

Channel Merah (R)
mean=129.3



Channel Warna Asli (Sebelum Manipulasi)
Channel Hijau (G)
mean=107.0



Channel Biru (B)
mean=99.6



[3] Memanipulasi channel (R+50, B-30)...

R: 129.3 → 178.8 (+50)

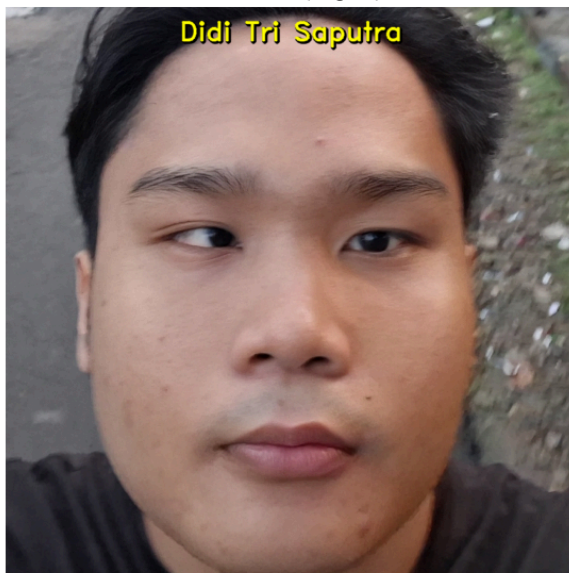
G: 107.0 → 107.0 (tidak berubah)

B: 99.6 → 70.2 (-30)

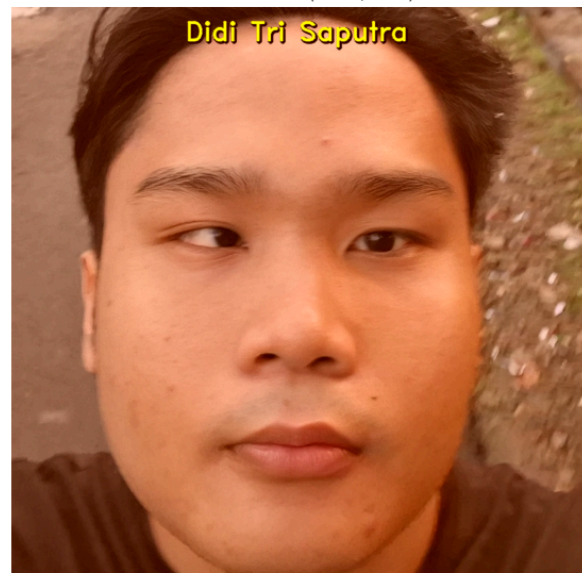
[4] Menggabungkan channel dan menyimpan...

Perbandingan Sebelum & Sesudah Manipulasi RGB

Gambar Asli (Original)

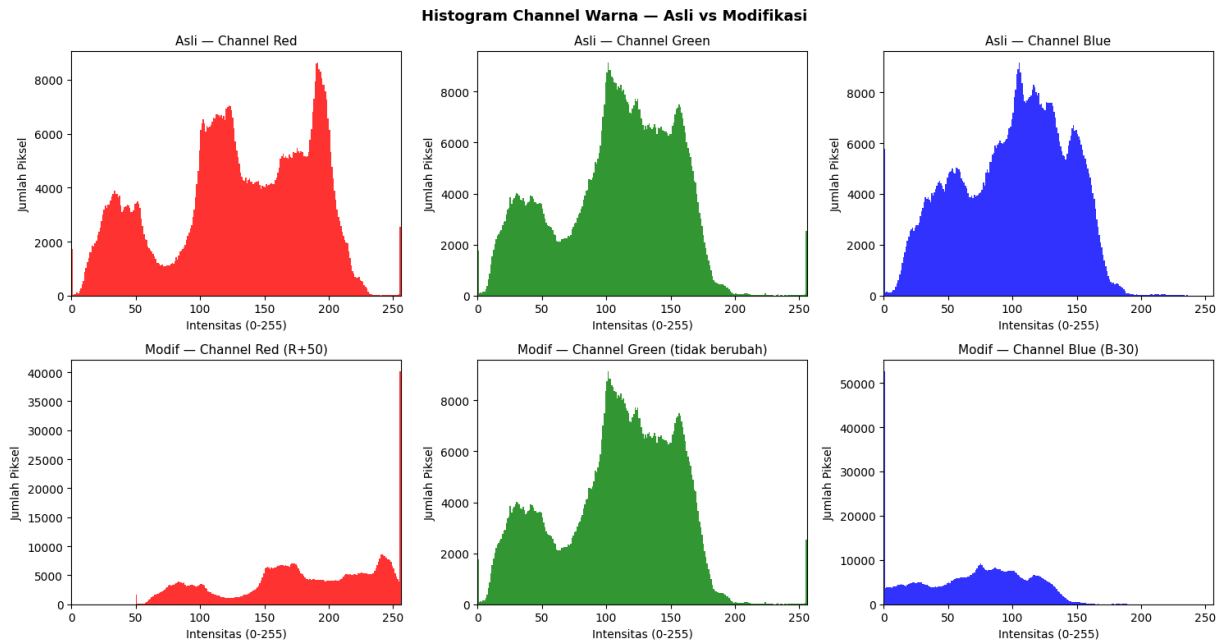


Hasil Modifikasi (R+50, B-30)



✓ Tersimpan di: C:\Users\HP\Pengolahan_Multimedia\hasil_modifikasi_rgb.png

[5] Menampilkan histogram per channel...



=====

SELESAI! File output:

=====

✓ C:\Users\HP\Pengolahan_Multimedia\hasil_modifikasi_rgb.png

✓ C:\Users\HP\Pengolahan_Multimedia\histogram_rgb.png

=====

Penjelasan — Dampak Manipulasi Channel RGB

Operasi yang Dilakukan

Channel	Operasi	Efek
Merah (R)	+50 (clip maks 255)	Intensitas merah naik
Hijau (G)	Tidak diubah	Tetap sama
Biru (B)	-30 (clip min 0)	Intensitas biru turun

Dampak Visual

- **R+50** → gambar tampak lebih **hangat/kemerahan**
- **B-30** → warna **kebiruan/dingin** berkurang
- Kombinasi keduanya → efek seperti filter **warm** pada aplikasi foto

Dampak pada Histogram

- Histogram **R** bergeser ke **kanan** (intensitas lebih tinggi)
- Histogram **B** bergeser ke **kiri** (intensitas lebih rendah)
- Histogram **G** tidak berubah

- Piksel R yang sudah 255 akan **clipping** → sedikit kehilangan detail di area terang

Soal 3 — Deteksi Tepi dan Filter Citra

- Ambil gambar **objek dengan background bertekstur** (misalnya kain bermotif, jerami, atau batu).
- Terapkan **edge detection (Canny)** dan tampilkan hasilnya.
- Lakukan **thresholding dengan nilai ambang tertentu** (bebas Anda tentukan) agar hanya objek utama yang tersisa.
- Buat **bounding box** di sekitar objek hasil segmentasi (boleh manual atau otomatis).
- Terapkan **filter blur** dan **filter sharpening**, lalu **bandingkan hasil keduanya**.
- Jelaskan bagaimana setiap filter memengaruhi detail gambar dalam format **Markdown**.

In [8]:

```
"""
=====
    Soal 3 – Deteksi Tepi dan Filter Citra
    Pengolahan Citra Digital
=====
    Foto yang dibutuhkan:
    Objek dengan background bertekstur
    (kain bermotif, jerami, batu, dll.)
=====
"""

import numpy as np
import matplotlib.pyplot as plt
import cv2
import os

PATH_FOTO = PATH_FOTO = r"C:\Users\HP\Pengolahan_Multimedia\data\soal 3.JPEG"
CANNY_THRESHOLD1 = 50
CANNY_THRESHOLD2 = 150
THRESH_VALUE = 127
BLUR_KERNEL = (15, 15)
SHARP_STRENGTH = 1.5

print("=" * 60)
print("  SOAL 3 – DETEKSI TEPI DAN FILTER CITRA")
print("=" * 60)

print("\n[1] Memuat gambar...")

img = cv2.imread(PATH_FOTO)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print(f"    Ukuran: {img_rgb.shape[1]}x{img_rgb.shape[0]} px")

plt.figure(figsize=(6, 5))
plt.imshow(img_rgb)
plt.title('Gambar Asli (Original)', fontsize=12)
plt.axis('off')
```



```

plt.tight_layout()
plt.show()

print(f"\n[2] Deteksi tepi dengan Canny (threshold1={CANNY_THRESHOLD1}, threshold2=
edges = cv2.Canny(img_gray, CANNY_THRESHOLD1, CANNY_THRESHOLD2)

print(f"    Piksel tepi terdeteksi: {np.sum(edges > 0)}")

fig, axs = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle('Deteksi Tepi – Canny Edge Detection', fontsize=13, fontweight='bold')

axs[0].imshow(img_rgb)
axs[0].set_title('Gambar Asli (Original)', fontsize=11)
axs[0].axis('off')

axs[1].imshow(edges, cmap='gray')
axs[1].set_title(f'Canny Edge\n(threshold1={CANNY_THRESHOLD1}, threshold2={CANNY_TH
axs[1].axis('off')

plt.tight_layout()
plt.savefig('hasil_canny.png', dpi=150, bbox_inches='tight')
plt.show()
print("    ✓ Tersimpan: hasil_canny.png")

print(f"\n[3] Thresholding dengan nilai ambang = {THRESH_VALUE}...")

ret, thresh_binary = cv2.threshold(img_gray, THRESH_VALUE, 255, cv2.THRESH_BINARY)
ret_otsu, thresh_otsu = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY + cv2.THR

print(f"    Nilai ambang manual : {THRESH_VALUE}")
print(f"    Nilai ambang Otsu    : {ret_otsu:.0f} (otomatis)")

fig, axs = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Thresholding – Isolasi Objek Utama', fontsize=13, fontweight='bold')

axs[0].imshow(img_gray, cmap='gray')
axs[0].set_title('Grayscale', fontsize=11)
axs[0].axis('off')

axs[1].imshow(thresh_binary, cmap='gray')
axs[1].set_title(f'Binary Threshold\n(nilai ambang = {THRESH_VALUE})', fontsize=11)
axs[1].axis('off')

axs[2].imshow(thresh_otsu, cmap='gray')
axs[2].set_title(f'Otsu Threshold\n(nilai ambang otomatis = {ret_otsu:.0f})', fonts
axs[2].axis('off')

plt.tight_layout()
plt.savefig('hasil_threshold.png', dpi=150, bbox_inches='tight')
plt.show()
print("    ✓ Tersimpan: hasil_threshold.png")

print("\n[4] Membuat bounding box di sekitar objek...")

contours, _ = cv2.findContours(thresh_otsu, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIM

```

```

contour_terbesar = max(contours, key=cv2.contourArea)
area_terbesar    = cv2.contourArea(contour_terbesar)

print(f"    Jumlah kontur terdeteksi : {len(contours)}")
print(f"    Area kontur terbesar      : {area_terbesar:.0f} px²")

x_bb, y_bb, w_bb, h_bb = cv2.boundingRect(contour_terbesar)
print(f"    Bounding box                : x={x_bb}, y={y_bb}, w={w_bb}, h={h_bb}")

img_bbox = img_rgb.copy()
cv2.drawContours(img_bbox, [contour_terbesar], -1, (0, 255, 0), 3)
cv2.rectangle(img_bbox, (x_bb, y_bb), (x_bb+w_bb, y_bb+h_bb), (255, 0, 0), 3)
cv2.putText(img_bbox, f'Objek Utama', (x_bb, y_bb - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 0), 2)

fig, axs = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle('Bounding Box Objek Utama', fontsize=13, fontweight='bold')

axs[0].imshow(thresh_otsu, cmap='gray')
axs[0].set_title('Hasil Thresholding (Otsu)', fontsize=11)
axs[0].axis('off')

axs[1].imshow(img_bbox)
axs[1].set_title('Bounding Box (Merah) + Kontur (Hijau)', fontsize=11)
axs[1].axis('off')

plt.tight_layout()
plt.savefig('hasil_bounding_box.png', dpi=150, bbox_inches='tight')
plt.show()
print("    ✓ Tersimpan: hasil_bounding_box.png")

print(f"\n[5] Menerapkan filter blur (kernel={BLUR_KERNEL})...")
img_blur = cv2.GaussianBlur(img_rgb, BLUR_KERNEL, 0)

print(f"    Kernel blur : {BLUR_KERNEL}")

print(f"\n[6] Menerapkan filter sharpening (strength={SHARP_STRENGTH})...")
kernel_sharp = np.array([[ 0, -1,  0],
                        [-1,  5, -1],
                        [ 0, -1,  0]], dtype=np.float32)

img_sharp = cv2.filter2D(img_rgb, -1, kernel_sharp)
img_blur_mask = cv2.GaussianBlur(img_rgb, (9, 9), 10)
img_unsharp = cv2.addWeighted(img_rgb, 1 + SHARP_STRENGTH,
                              img_blur_mask, -SHARP_STRENGTH, 0)

print("\n[7] Membandingkan hasil filter...")

fig, axs = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Perbandingan Filter – Original | Blur | Sharpening', fontsize=13, fontweight='bold')

axs[0].imshow(img_rgb)
axs[0].set_title('Original', fontsize=11)
axs[0].axis('off')

axs[1].imshow(img_blur)

```

```

axs[1].set_title(f'Gaussian Blur\n(kernel={BLUR_KERNEL})', fontsize=11)
axs[1].axis('off')

axs[2].imshow(img_unsharp)
axs[2].set_title(f'Sharpening (Unsharp Mask)\n(strength={SHARP_STRENGTH})', fontsize=11)
axs[2].axis('off')

plt.tight_layout()
plt.savefig('hasil_filter.png', dpi=150, bbox_inches='tight')
plt.show()
print("    ✓ Tersimpan: hasil_filter.png")

print("\n[8] Perbandingan detail tekstur (crop area tengah)...")

# Crop area tengah untuk melihat detail tekstur lebih jelas
cy, cx = img_rgb.shape[0] // 2, img_rgb.shape[1] // 2
crop_sz = 150

detail_ori = img_rgb [cy-crop_sz:cy+crop_sz, cx-crop_sz:cx+crop_sz]
detail_blur = img_blur [cy-crop_sz:cy+crop_sz, cx-crop_sz:cx+crop_sz]
detail_sharp = img_unsharp[cy-crop_sz:cy+crop_sz, cx-crop_sz:cx+crop_sz]

fig, axs = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Detail Tekstur – Crop Area Tengah', fontsize=13, fontweight='bold')

axs[0].imshow(detail_ori)
axs[0].set_title('Original (Detail)', fontsize=11)
axs[0].axis('off')

axs[1].imshow(detail_blur)
axs[1].set_title('Blur (Detail)', fontsize=11)
axs[1].axis('off')

axs[2].imshow(detail_sharp)
axs[2].set_title('Sharpening (Detail)', fontsize=11)
axs[2].axis('off')

plt.tight_layout()
plt.savefig('hasil_filter_detail.png', dpi=150, bbox_inches='tight')
plt.show()
print("    ✓ Tersimpan: hasil_filter_detail.png")

print("\n" + "=" * 60)
print("  SELESAI! File output:")
print("=" * 60)
for f in ['hasil_canny.png', 'hasil_threshold.png',
          'hasil_bounding_box.png', 'hasil_filter.png',
          'hasil_filter_detail.png']:
    print(f"    ✓ {os.path.join(os.getcwd(), f)}")
print("=" * 60)

```

=====

SOAL 3 – DETEKSI TEPI DAN FILTER CITRA

[1] Memuat gambar...
Ukuran: 1600x1200 px

Gambar Asli (Original)

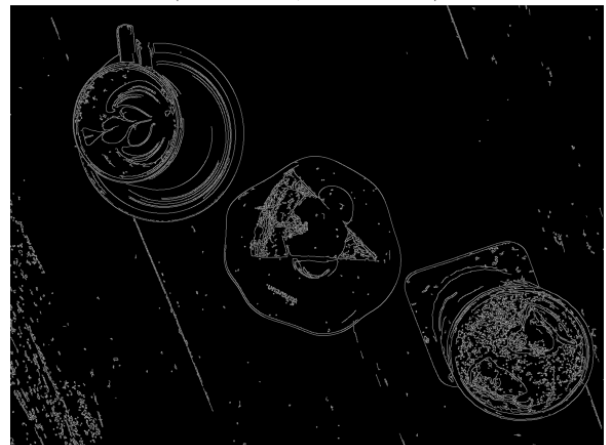


[2] Deteksi tepi dengan Canny (threshold1=50, threshold2=150)...
Piksel tepi terdeteksi: 65427

Gambar Asli (Original)

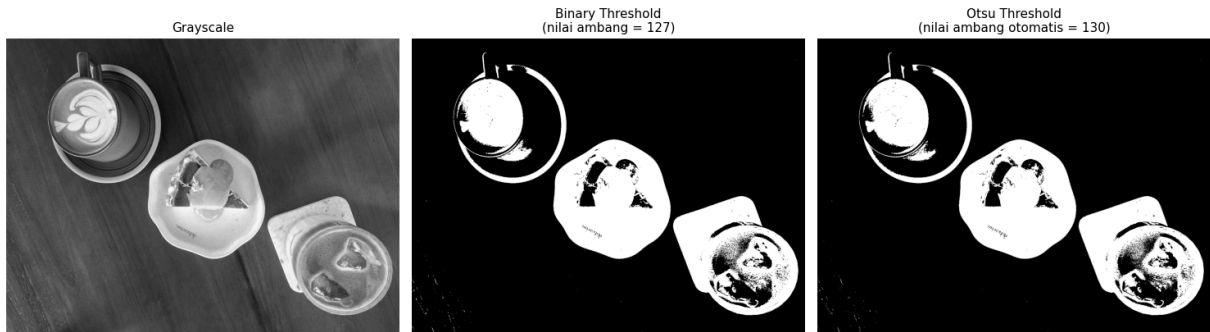
Deteksi Tepi — Canny Edge Detection

Canny Edge
(threshold1=50, threshold2=150)



✓ Tersimpan: hasil_canny.png

[3] Thresholding dengan nilai ambang = 127...
Nilai ambang manual : 127
Nilai ambang Otsu : 130 (otomatis)

Thresholding — Isolasi Objek Utama

✓ Tersimpan: hasil_threshold.png

[4] Membuat bounding box di sekitar objek...

Jumlah kontur terdeteksi : 204

Area kontur terbesar : 172966 px²

Bounding box : x=576, y=410, w=477, h=486

Bounding Box Objek Utama

✓ Tersimpan: hasil_bounding_box.png

[5] Menerapkan filter blur (kernel=(15, 15))...

Kernel blur : (15, 15)

[6] Menerapkan filter sharpening (strength=1.5)...

[7] Membandingkan hasil filter...

Perbandingan Filter — Original | Blur | Sharpening

✓ Tersimpan: hasil_filter.png

[8] Perbandingan detail tekstur (crop area tengah)...



✓ Tersimpan: hasil_filter_detail.png

=====

SELESAI! File output:

=====

✓ C:\Users\HP\Pengolahan_Multimedia\hasil_canny.png
 ✓ C:\Users\HP\Pengolahan_Multimedia\hasil_threshold.png
 ✓ C:\Users\HP\Pengolahan_Multimedia\hasil_bounding_box.png
 ✓ C:\Users\HP\Pengolahan_Multimedia\hasil_filter.png
 ✓ C:\Users\HP\Pengolahan_Multimedia\hasil_filter_detail.png

=====

Penjelasan — Soal 3: Deteksi Tepi dan Filter Citra

1. Canny Edge Detection

Canny adalah algoritma deteksi tepi yang bekerja dalam beberapa tahap:

1. **Noise Reduction** — gambar dihaluskan dengan Gaussian filter
2. **Gradient Calculation** — menghitung intensitas perubahan piksel di setiap arah
3. **Non-Maximum Suppression** — menipiskan tepi menjadi satu piksel
4. **Double Thresholding** — memisahkan tepi kuat dan lemah berdasarkan dua nilai ambang
5. **Edge Tracking** — tepi lemah yang terhubung ke tepi kuat dipertahankan

Parameter:

- `threshold1` (ambang bawah) → tepi dengan gradien di bawah ini diabaikan
- `threshold2` (ambang atas) → tepi dengan gradien di atas ini pasti dianggap tepi

2. Thresholding

Thresholding mengubah gambar grayscale menjadi gambar biner (hitam-putih) berdasarkan nilai ambang tertentu.

Metode	Cara	Kelebihan
Binary Manual	Piksel > ambang → putih, lainnya → hitam	Sederhana, mudah dikontrol
Otsu	Nilai ambang dihitung otomatis dari histogram	Adaptif, tidak perlu trial-error

Thresholding digunakan untuk mengisolasi objek utama dari background, sehingga kontur dan bounding box bisa dideteksi.

3. Bounding Box

Bounding box adalah kotak persegi minimum yang mengelilingi objek hasil segmentasi. Dibuat menggunakan `cv2.findContours()` untuk menemukan kontur, lalu `cv2.boundingRect()` untuk mendapatkan koordinat kotak pembatasnya.

4. Filter Blur vs Sharpening

Gaussian Blur

Filter blur menghaluskan gambar dengan cara merata-ratakan nilai piksel di sekitar setiap piksel menggunakan kernel Gaussian.

Efek:

- Mengurangi noise dan detail halus
- Tepi objek menjadi lebih lembut/kabur
- Detail tekstur pada background berkurang
- Sering digunakan sebagai preprocessing sebelum deteksi tepi

Sharpening (Unsharp Masking)

Filter sharpening mempertajam gambar dengan cara memperkuat perbedaan intensitas antar piksel yang berdekatan.

Efek:

- Detail dan tepi objek menjadi lebih tajam dan jelas
- Tekstur pada background lebih terlihat
- Noise pada gambar ikut diperkuat jika gambar sudah berisik
- Berguna untuk meningkatkan keterbacaan detail halus

Perbandingan

	Blur	Sharpening
Detail	Berkurang	Bertambah
Tepi	Melebar/kabur	Lebih tajam
Noise	Berkurang	Bertambah
Kegunaan	Preprocessing, reduksi noise	Peningkatan kualitas visual

Dibuat menggunakan Python · OpenCV · NumPy · Matplotlib

Soal 4 — Deteksi Wajah dan Filter Digital Kreatif

- Ambil gambar diri Anda dengan ekspresi wajah **netral**.
- Lakukan **deteksi wajah dan landmark** menggunakan salah satu dari:
 - **MediaPipe**, atau
 - **Dlib**, atau
 - **OpenCV**.
- Buat **overlay filter digital kreatif** karya Anda sendiri, misalnya:
 - topi, kumis, masker, helm, aksesoris, atau bentuk unik lainnya.
 - Filter boleh dibuat dari **gambar eksternal (PNG)** atau digambar langsung (misal bentuk lingkaran, garis, poligon, dll).
- Pastikan posisi overlay menyesuaikan **landmark wajah** dengan logis.
- **Gunakan alpha blending sebagai saran** agar hasil tampak lebih natural.
- Tampilkan perbandingan antara **gambar asli** dan **hasil dengan filter**.
- Jelaskan bagaimana Anda menghitung posisi overlay dan tantangan yang dihadapi selama implementasi (gunakan **Markdown**).

```
In [9]: """
=====
    Soal 4 – Deteksi Wajah dan Filter Digital Kreatif
    Pengolahan Citra Digital
=====
"""

import numpy as np
import matplotlib.pyplot as plt
import cv2
import os

PATH_FOTO      = r"C:\Users\HP\Pengolahan_Multimedia\data\soal 4.jpeg"
PATH_OVERLAY   = r"C:\Users\HP\Pengolahan_Multimedia\data\glasses2.png"  # kacamata
```

```

print("=" * 60)
print("  SOAL 4 – DETEKSI WAJAH DAN FILTER DIGITAL KREATIF")
print("=" * 60)

print("\n[1] Memuat gambar...")
img      = cv2.imread(PATH_FOTO)
img_rgb  = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
print(f"    Ukuran: {img_rgb.shape[1]}x{img_rgb.shape[0]} px")

plt.figure(figsize=(5,7))
plt.imshow(img_rgb); plt.title('Gambar Asli'); plt.axis('off')
plt.tight_layout(); plt.show()

print("\n[2] Deteksi wajah dengan Haar Cascade...")
img_gray  = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
faces = face_cascade.detectMultiScale(img_gray, 1.1, 5, minSize=(100,100))
print(f"    Wajah terdeteksi: {len(faces)}")
x_f, y_f, w_f, h_f = faces[0]
print(f"    BBox: x={x_f}, y={y_f}, w={w_f}, h={h_f}")

print("\n[3] Estimasi landmark wajah...")
cx      = x_f + w_f // 2
eye_y   = y_f + int(h_f * 0.38)

landmark = {
    'Mata Kiri' : (x_f + int(w_f*0.30), y_f + int(h_f*0.25)),
    'Mata Kanan': (x_f + int(w_f*0.70), y_f + int(h_f*0.25)),
    'Hidung'    : (cx, y_f + int(h_f*0.50)),
    'Bibir'     : (cx, y_f + int(h_f*0.68)),
}
lm_colors = [(255,50,50),(255,50,50),(50,100,255),(255,165,0)]
for (label,(px,py)),col in zip(landmark.items(),lm_colors):
    print(f"    {label:12s}: ({px}, {py})")

img_bbox = img_rgb.copy()
cv2.rectangle(img_bbox,(x_f,y_f),(x_f+w_f,y_f+h_f),(0,255,0),4)
cv2.putText(img_bbox,'Wajah',(x_f,y_f-15),cv2.FONT_HERSHEY_SIMPLEX,1.2,(0,255,0),3)
for (label,(px,py)),col in zip(landmark.items(),lm_colors):
    cv2.circle(img_bbox,(px,py),14,col,-1)
    cv2.putText(img_bbox,label,(px+15,py+6),cv2.FONT_HERSHEY_SIMPLEX,0.8,col,2)

plt.figure(figsize=(5,7))
plt.imshow(img_bbox)
plt.title('Deteksi Wajah + Landmark\n(OpenCV Haar Cascade)',fontsize=12)
plt.axis('off'); plt.tight_layout(); plt.show()

print("\n[4] Menyiapkan overlay kaca mata...")

g = cv2.imread(PATH_OVERLAY, cv2.IMREAD_UNCHANGED)
print(f"    Channels: {g.shape}")

if g.shape[2] == 4:
    g_bgr = g[:, :, :3]
    m_full = g[:, :, 3]

```

```

else:
    g_bgr = g
    gray_g = cv2.cvtColor(g, cv2.COLOR_BGR2GRAY)
    _, m_full = cv2.threshold(gray_g, 15, 255, cv2.THRESH_BINARY)

    coords = cv2.findNonZero(m_full)
    xk,yk,wk,hk = cv2.boundingRect(coords)
    g_crop = cv2.cvtColor(g_bgr[yk:yk+hk, xk:xk+wk], cv2.COLOR_BGR2RGB)
    m_crop = m_full[yk:yk+hk, xk:xk+wk]

    tw = int(w_f * 0.85)
    th = int(hk * tw / wk)
    g_r = cv2.resize(g_crop, (tw, th), interpolation=cv2.INTER_LINEAR)
    m_r = cv2.resize(m_crop, (tw, th), interpolation=cv2.INTER_NEAREST)
    _, m_r = cv2.threshold(m_r, 127, 255, cv2.THRESH_BINARY)

    print(f"    Ukuran kacamata: {tw}x{th} px")
    print(f"    Piksel opak    : {np.sum(m_r > 0)}")

    print("\n[5] Menghitung posisi overlay...")
    ox = cx - tw//2
    oy = eye_y - th//2
    print(f"    Posisi: ox={ox}, oy={oy}")

    img_bbox_pos = img_bbox.copy()
    cv2.rectangle(img_bbox_pos, (ox,oy), (ox+tw,oy+th), (255,255,0), 2)
    cv2.putText(img_bbox_pos, 'Area Kacamata', (ox,oy-12), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255,255,0))
    plt.figure(figsize=(5,7))
    plt.imshow(img_bbox_pos)
    plt.title('Posisi Overlay Kacamata\n(Kuning = Area Penempatan)', fontsize=12)
    plt.axis('off'); plt.tight_layout(); plt.show()

    print("\n[6] Menerapkan alpha blending...")
    result = img_rgb.copy()
    roi = result[oy:oy+th, ox:ox+tw]
    alpha = m_r.astype(float)/255.0
    a3 = np.stack([alpha]*3, axis=2)
    blended = (g_r*a3 + roi*(1-a3)).astype(np.uint8)
    result[oy:oy+th, ox:ox+tw] = blended
    print("    ✓ Formula: pixel_out = α·kacamata + (1-α)·background")

    print("\n[7] Menampilkan perbandingan...")
    fig,axs = plt.subplots(1,3,figsize=(18,7))
    fig.suptitle('Soal 4 – Deteksi Wajah dan Filter Digital Kreatif', fontsize=14, fontweight='bold')
    axs[0].imshow(img_rgb); axs[0].set_title('Gambar Asli (Original)', fontsize=12);
    axs[1].imshow(img_bbox); axs[1].set_title('Deteksi Wajah + Landmark\n(OpenCV Haar Cascade)', fontsize=12);
    axs[2].imshow(result); axs[2].set_title('Hasil Filter Kreatif\n(Overlay Kacamata)', fontsize=12);
    plt.tight_layout()
    plt.savefig('hasil_filter_kreatif.png', dpi=150, bbox_inches='tight')
    plt.show()

    cv2.imwrite('hasil_wajah_bbox.png', cv2.cvtColor(img_bbox, cv2.COLOR_RGB2BGR))
    cv2.imwrite('hasil_filter_kreatif.png', cv2.cvtColor(result, cv2.COLOR_RGB2BGR))

    print(f"\n{'='*60}")
    print("    SELESAI! File output:")

```



```
print(f"{' '*60}")  
print(f"  ✓ {os.path.join(os.getcwd(), 'hasil_wajah_bbox.png')}")  
print(f"  ✓ {os.path.join(os.getcwd(), 'hasil_filter_kreatif.png')}")
```

=====

SOAL 4 – DETEKSI WAJAH DAN FILTER DIGITAL KREATIF

=====

[1] Memuat gambar...
Ukuran: 1194x1592 px

Gambar Asli



[2] Deteksi wajah dengan Haar Cascade...

Wajah terdeteksi: 1

BBox: x=350, y=728, w=543, h=543

[3] Estimasi landmark wajah...

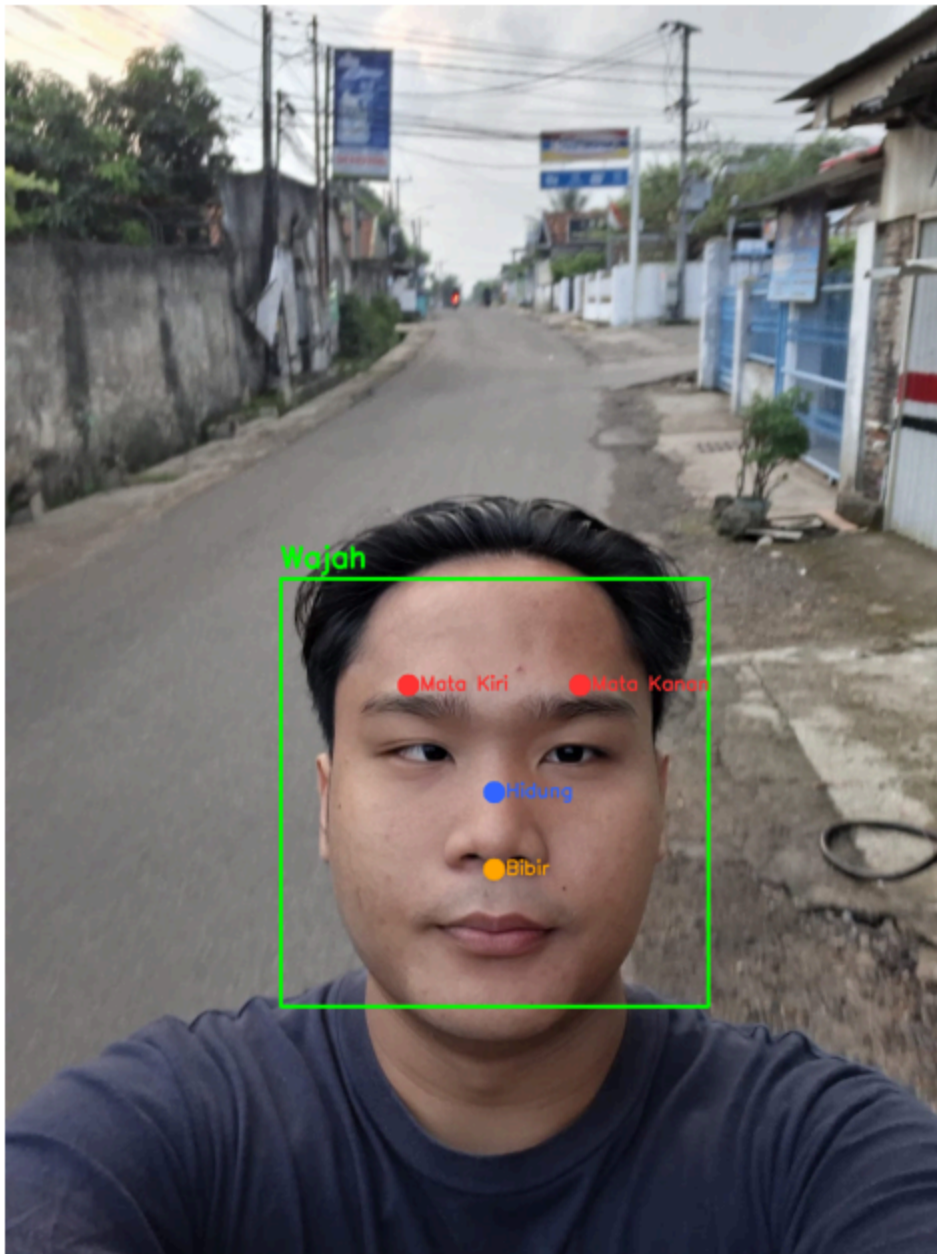
Mata Kiri : (512, 863)

Mata Kanan : (730, 863)

Hidung : (621, 999)

Bibir : (621, 1097)

Deteksi Wajah + Landmark (OpenCV Haar Cascade)



[4] Menyiapkan overlay kaca mata...

Channels: (290, 835, 4)

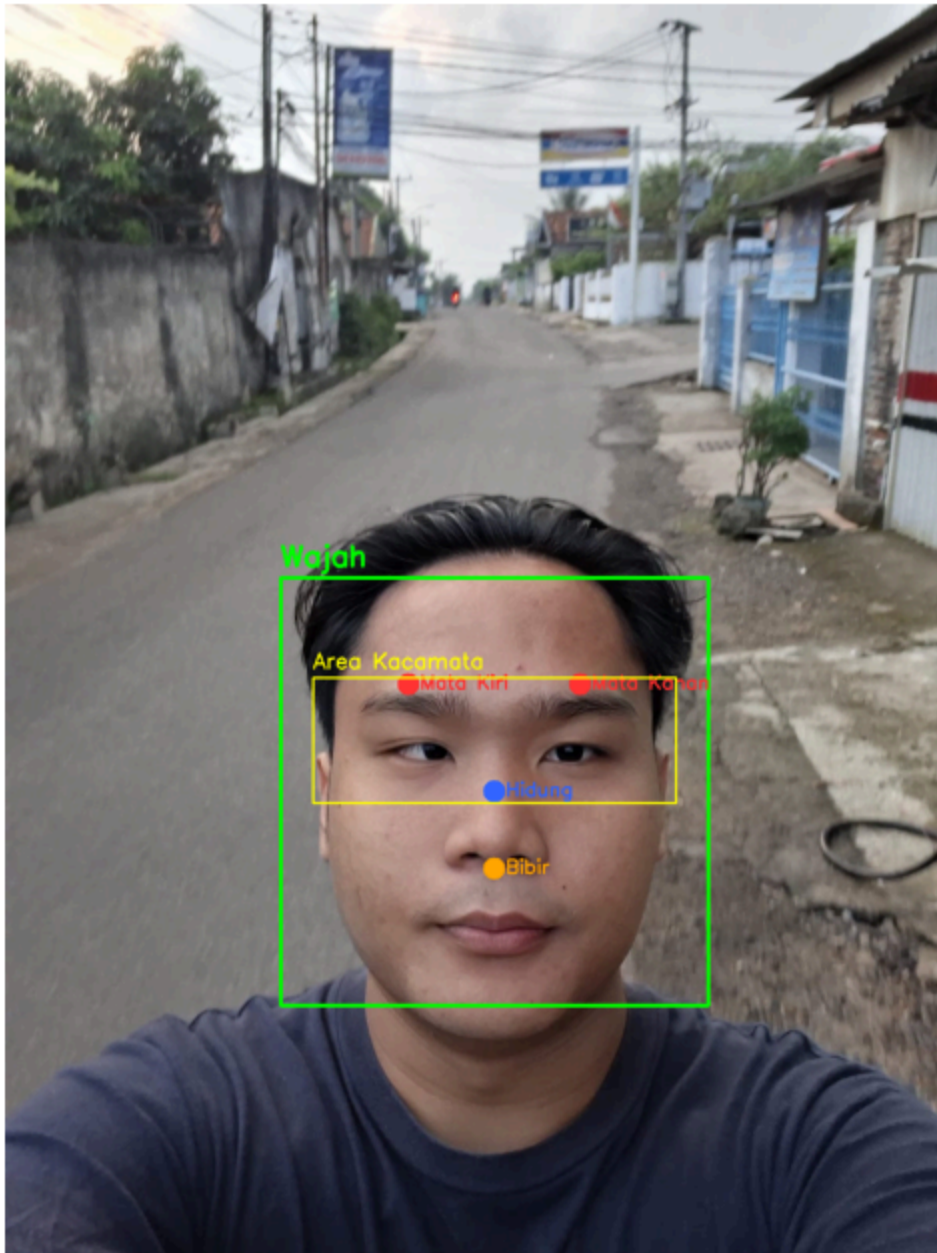
Ukuran kaca mata: 461x159 px

Piksel opak : 52634

[5] Menghitung posisi overlay...

Posisi: ox=391, oy=855

Posisi Overlay Kacamata (Kuning = Area Penempatan)



[6] Menerapkan alpha blending...

✓ Formula: $\text{pixel_out} = \alpha \cdot \text{kacamata} + (1 - \alpha) \cdot \text{background}$

[7] Menampilkan perbandingan...

Soal 4 — Deteksi Wajah dan Filter Digital Kreatif

=====

SELESAI! File output:

=====

- ✓ C:\Users\HP\Pengolahan_Multimedia\hasil_wajah_bbox.png
- ✓ C:\Users\HP\Pengolahan_Multimedia\hasil_filter_kreatif.png

Penjelasan — Soal 4: Deteksi Wajah dan Filter Digital Kreatif

1. Metode Deteksi Wajah — OpenCV Haar Cascade

Deteksi wajah menggunakan **Haar Cascade Classifier** dari OpenCV, yaitu model machine learning yang dilatih untuk mengenali pola wajah berdasarkan fitur-fitur Haar (terang-gelap pada area mata, hidung, mulut).

Parameter yang digunakan:

Parameter	Nilai	Fungsi
scaleFactor	1.1	Skala pencarian wajah di setiap iterasi
minNeighbors	5	Jumlah tetangga minimum agar dianggap wajah
minSize	(100,100)	Ukuran wajah minimum yang dicari

2. Cara Menghitung Posisi Overlay

Karena hanya menggunakan Haar Cascade (tanpa landmark detector khusus), posisi landmark diestimasi secara proporsional dari bounding box wajah:

Soal 5 — Perspektif dan Peningkatan Kualitas Citra

- Ambil gambar **objek datar** seperti karya tangan di kertas, tulisan di papan tulis, atau foto produk di meja dengan kondisi pencahayaan atau sudut yang tidak ideal.
- Lakukan **preprocessing** untuk memperbaiki tampilan agar lebih rapi dan jelas, dengan langkah-langkah:
 - Konversi ke **grayscale**.
 - **Koreksi perspektif (transformasi homografi)** menggunakan **4 titik manual** agar objek terlihat sejajar dan tidak terdistorsi.
 - Terapkan **thresholding adaptif atau Otsu** (pilih salah satu, dan jelaskan alasan pilihan Anda).
- Tampilkan **setiap tahap pemrosesan dalam satu grid** agar mudah dibandingkan.
- Jelaskan fungsi masing-masing tahap dan bagaimana teknik ini meningkatkan kualitas visual citra (gunakan **Markdown**).

```
In [10]: """
=====
Soal 5 – Perspektif dan Peningkatan Kualitas Citra
Pengolahan Citra Digital
=====
"""

import numpy as np
import matplotlib.pyplot as plt
import cv2
import os

PATH_FOTO = r"C:\Users\HP\Pengolahan_Multimedia\data\soal 5.jpeg"

SRC_PTS = np.array([
    [50, 30 ],
    [3060, 80 ],
    [3060, 2250],
    [20, 2220],
], dtype=np.float32)
DST_W, DST_H = 2000, 1400

print("=" * 60)
print(" SOAL 5 – PERSPEKTIF DAN PENINGKATAN KUALITAS CITRA")
print("=" * 60)

print("\n[1] Memuat gambar...")

img = cv2.imread(PATH_FOTO)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

print(f"    Ukuran: {img_rgb.shape[1]}x{img_rgb.shape[0]} px")

print("\n[2] Konversi ke grayscale...")
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(f"    Shape grayscale: {img_gray.shape}")
```

```

print("\n[3] Koreksi perspektif dengan transformasi homografi...")
dst_pts = np.array([
    [0, 0],
    [DST_W, 0],
    [DST_W, DST_H],
    [0, DST_H],
], dtype=np.float32)
M = cv2.getPerspectiveTransform(SRC_PTS, dst_pts)
print(f"    Matriks Homografi (3x3):\n{M.round(4)}")
warped_gray = cv2.warpPerspective(img_gray, M, (DST_W, DST_H))
warped_rgb = cv2.warpPerspective(img_rgb, M, (DST_W, DST_H))

print(f"    Ukuran setelah koreksi: {DST_W}x{DST_H} px")

print("\n[4] Thresholding Otsu...")
ret_otsu, thresh_otsu = cv2.threshold(
    warped_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
print(f"    Nilai ambang Otsu (otomatis): {ret_otsu:.0f}")

print("\n[5] Thresholding adaptif (untuk perbandingan)...")

thresh_adapt = cv2.adaptiveThreshold(
    warped_gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    blockSize=31,
    C=10
)
print(f"    Metode: Adaptive Gaussian (blockSize=31, C=10)")

print("\n[6] Menampilkan semua tahap dalam grid...")
GW, GH = 900, 630

def rz(im, cmap_flag=False):
    return cv2.resize(im, (GW, GH))
img_pts = img_rgb.copy()
scale_x = img_rgb.shape[1] / 3120
scale_y = img_rgb.shape[0] / 4160
for i, (x, y) in enumerate(SRC_PTS):
    px, py = int(x * scale_x), int(y * scale_y)
    cv2.circle(img_pts, (px, py), 40, (255, 0, 0), -1)
    cv2.putText(img_pts, f'P{i+1}', (px+15, py-15),
                cv2.FONT_HERSHEY_SIMPLEX, 2.5, (255, 255, 0), 5)
cv2.polylines(img_pts,
    [np.array([(int(x*scale_x), int(y*scale_y)) for x,y in SRC_PTS], dtype=np.int32
    True, (0, 255, 0), 8)

# Grid 2x3
fig, axs = plt.subplots(2, 3, figsize=(18, 11))
fig.suptitle('Soal 5 – Perspektif dan Peningkatan Kualitas Citra\n(Setiap Tahap Pem
    fontsize=14, fontweight='bold')

axs[0,0].imshow(rz(img_rgb))
axs[0,0].set_title('① Gambar Asli (Original)', fontsize=11, fontweight='bold')

```



```

axs[0,0].axis('off')

axs[0,1].imshow(rz(img_pts))
axs[0,1].set_title('@ 4 Titik Manual Koreksi Perspektif\n(Merah=titik, Hijau=area p
axs[0,1].axis('off')

axs[0,2].imshow(rz(img_gray), cmap='gray')
axs[0,2].set_title('Ⓢ Konversi Grayscale', fontsize=11, fontweight='bold')
axs[0,2].axis('off')

axs[1,0].imshow(rz(warped_rgb))
axs[1,0].set_title('@ Setelah Koreksi Perspektif\n(Transformasi Homografi)', fontsi
axs[1,0].axis('off')

axs[1,1].imshow(rz(thresh_otsu), cmap='gray')
axs[1,1].set_title(f'Ⓢ Thresholding Otsu\n(Nilai ambang otomatis = {ret_otsu:.0f})'
axs[1,1].axis('off')

axs[1,2].imshow(rz(thresh_adapt), cmap='gray')
axs[1,2].set_title('Ⓢ Thresholding Adaptif Gaussian\n(blockSize=31, C=10)', fontsiz
axs[1,2].axis('off')

plt.tight_layout()
plt.savefig('hasil_soal5_grid.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"    ✓ Tersimpan: {os.path.join(os.getcwd(), 'hasil_soal5_grid.png')}")

print("\n[7] Menyimpan file output...")

cv2.imwrite('soal5_grayscale.png', img_gray)
cv2.imwrite('soal5_warped.png', cv2.cvtColor(warped_rgb, cv2.COLOR_RGB2BGR))
cv2.imwrite('soal5_otsu.png', thresh_otsu)
cv2.imwrite('soal5_adaptive.png', thresh_adapt)

for f in ['soal5_grayscale.png', 'soal5_warped.png', 'soal5_otsu.png', 'soal5_adaptive
    print(f"    ✓ {os.path.join(os.getcwd(), f)}")

print("\n" + "=" * 60)
print("    SELESAI!")
print("=" * 60)

```

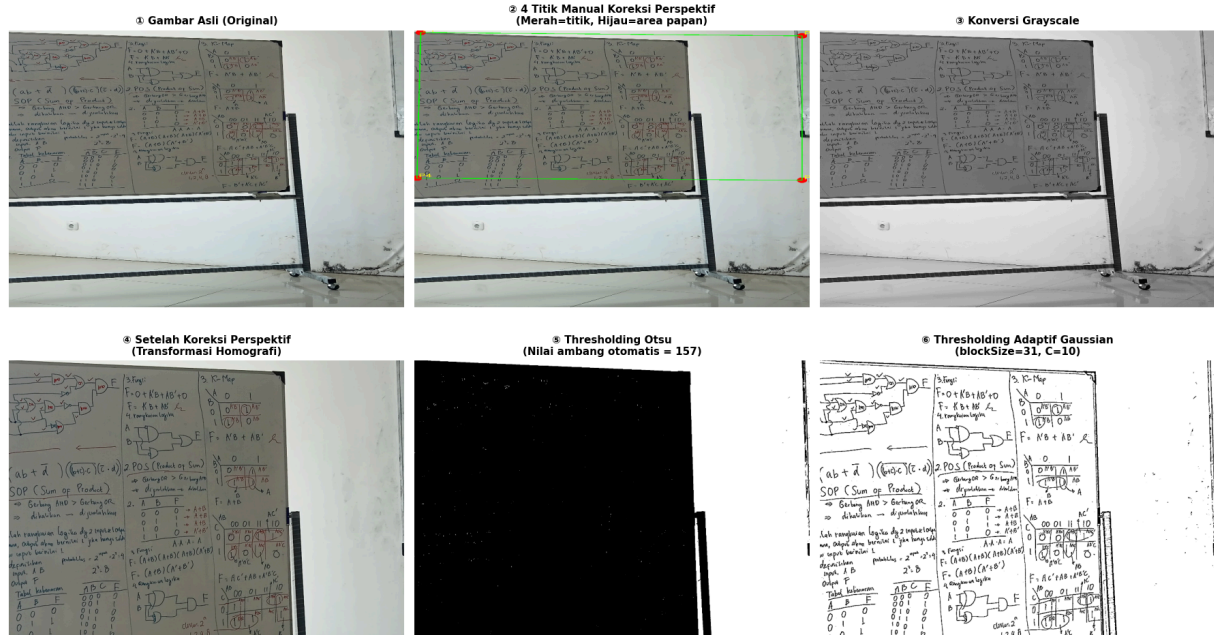
=====

SOAL 5 – PERSPEKTIF DAN PENINGKATAN KUALITAS CITRA

=====

- [1] Memuat gambar...
Ukuran: 3120x4160 px
- [2] Konversi ke grayscale...
Shape grayscale: (4160, 3120)
- [3] Koreksi perspektif dengan transformasi homografi...
Matriks Homografi (3x3):
[[6.58100e-01 9.00000e-03 -3.31770e+01]
[-1.07000e-02 6.45500e-01 -1.88283e+01]
[-0.00000e+00 0.00000e+00 1.00000e+00]]
Ukuran setelah koreksi: 2000x1400 px
- [4] Thresholding Otsu...
Nilai ambang Otsu (otomatis): 157
- [5] Thresholding adaptif (untuk perbandingan)...
Metode: Adaptive Gaussian (blockSize=31, C=10)
- [6] Menampilkan semua tahap dalam grid...

Soal 5 – Perspektif dan Peningkatan Kualitas Citra
(Setiap Tahap Pemrosesan)



✓ Tersimpan: C:\Users\HP\Pengolahan_Multimedia\hasil_soal5_grid.png

- [7] Menyimpan file output...
- ✓ C:\Users\HP\Pengolahan_Multimedia\soal5_grayscale.png
 - ✓ C:\Users\HP\Pengolahan_Multimedia\soal5_warped.png
 - ✓ C:\Users\HP\Pengolahan_Multimedia\soal5_otsu.png
 - ✓ C:\Users\HP\Pengolahan_Multimedia\soal5_adaptive.png

=====

SELESAI!

=====

Soal 5 — Perspektif dan Peningkatan Kualitas Citra

Deskripsi

Program ini memproses gambar papan tulis melalui beberapa tahap: koreksi perspektif, konversi grayscale, dan thresholding untuk meningkatkan kualitas citra.

Library yang Digunakan

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
import os
```

Konfigurasi

```
PATH_FOTO = r"D:\Pengolahan_Multimedia-main\data\soal 5.jpeg"
```

```
# 4 Titik sudut papan tulis [kiri_atas, kanan_atas, kanan_bawah, kiri_bawah]
```

```
SRC_PTS = np.array([
    [50,    30 ],
    [3060,  80 ],
    [3060, 2250],
    [20,    2220],
], dtype=np.float32)
```

```
# Ukuran output setelah koreksi perspektif
```

```
DST_W, DST_H = 2000, 1400
```

Yang Bisa Diubah

Parameter	Nilai Default	Keterangan
PATH_FOTO	soal 5.jpeg	Ganti dengan path gambar lain
SRC_PTS	4 titik sudut	Sesuaikan dengan posisi sudut objek di foto
DST_W, DST_H	2000 x 1400	Ukuran output hasil koreksi

Langkah-langkah Pemrosesan

Langkah 1 — Load Gambar

```
img = cv2.imread(PATH_FOTO)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Membaca gambar dari disk dan mengkonversi dari BGR (format OpenCV) ke RGB (format Matplotlib).

Langkah 2 — Konversi Grayscale

```
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Mengubah gambar berwarna menjadi grayscale untuk mempermudah proses thresholding.

Langkah 3 — Koreksi Perspektif (Homografi)

```
M = cv2.getPerspectiveTransform(SRC_PTS, dst_pts)
warped_gray = cv2.warpPerspective(img_gray, M, (DST_W, DST_H))
warped_rgb = cv2.warpPerspective(img_rgb, M, (DST_W, DST_H))
```

Menghitung matriks homografi dari 4 titik sumber ke 4 titik tujuan, lalu menerapkan transformasi perspektif sehingga gambar terlihat lurus/tegak.

Yang Bisa Diubah

```
DST_W, DST_H = 2000, 1400 # coba 1600x1200 atau 2400x1600
```

Langkah 4 — Thresholding Otsu

```
ret_otsu, thresh_otsu = cv2.threshold(
    warped_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Otsu secara **otomatis** menemukan nilai ambang optimal berdasarkan histogram gambar. Nilai ambang tidak perlu ditentukan manual.

Langkah 5 — Thresholding Adaptif

```
thresh_adapt = cv2.adaptiveThreshold(
    warped_gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    blockSize=31,
    C=10
)
```

Thresholding adaptif menggunakan nilai ambang yang berbeda untuk setiap area gambar, cocok untuk gambar dengan pencahayaan tidak merata.

Yang Bisa Diubah

Parameter	Nilai Default	Efek
blockSize	31	Ukuran area lokal (harus ganjil). Kecil = detail lebih tajam
C	10	Konstanta pengurangan. Besar = lebih banyak area putih
Metode	ADAPTIVE_THRESH_GAUSSIAN_C	Ganti ke ADAPTIVE_THRESH_MEAN_C

Langkah 6 — Visualisasi Grid 2x3

Menampilkan 6 tahap pemrosesan dalam satu figure:

Panel	Isi
①	Gambar asli
②	4 titik koreksi perspektif
③	Grayscale
④	Hasil koreksi perspektif
⑤	Thresholding Otsu
⑥	Thresholding Adaptif

Yang Bisa Diubah

```

GW, GH = 900, 630      # ukuran setiap panel di grid
figsize=(18, 11)        # ukuran total figure
dpi=150                 # kualitas gambar yang disimpan

```

Langkah 7 — Simpan Output

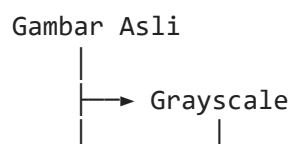
Program menyimpan 4 file hasil:

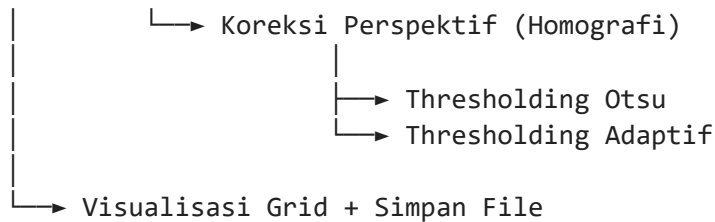
```

soal5_grayscale.png → hasil konversi grayscale
soal5_warped.png   → hasil koreksi perspektif
soal5_otsu.png      → hasil thresholding Otsu
soal5_adaptive.png  → hasil thresholding Adaptif

```

Ringkasan Alur Program





Aturan Umum Pengerjaan

- Kerjakan secara **mandiri**.
- Bantuan AI (seperti ChatGPT, Copilot, dsb.) diperbolehkan **dengan bukti percakapan** (screenshot / link / script percakapan).
- Source code antar mahasiswa harus berbeda.
- Jika mendapat bantuan teman, tuliskan nama dan NIM teman yang membantu.
- Plagiarisme akan dikenakan sanksi sesuai aturan akademik ITERA.
- Cantumkan seluruh **credit dan referensi** yang digunakan di bagian akhir notebook.
- Penjelasan setiap soal ditulis dalam **Markdown**, bukan di dalam komentar kode.

Aturan Pengumpulan

- Semua file kerja Anda (notebook `.ipynb`, gambar, dan hasil) **wajib diunggah ke GitHub repository tugas sebelumnya**.

- Gunakan struktur folder berikut di dalam repo Anda:

```

>Nama_NIM_Repo/ # Nama repo sebelumnya
├── assets_ws4/   # berisi semua gambar atau video asli (input)
├── results_ws4/  # berisi semua hasil modifikasi dan output
├── worksheet4.ipynb
└── NIM_Worksheet4.pdf
  
```

- File yang dikumpulkan ke **Tally** hanya berupa **hasil PDF** dari notebook Anda, dengan format nama:

NIM_Worksheet4.pdf

- Pastikan notebook telah dijalankan penuh sebelum diekspor ke PDF.
- Sertakan tautan ke repository GitHub Anda di bagian atas notebook atau di halaman pertama PDF.

✓ Catatan Akhir

Worksheet 4 ini bertujuan mengasah pemahaman Anda tentang manipulasi citra digital secara praktis. Gunakan kreativitas Anda untuk menghasilkan hasil visual yang menarik dan penjelasan konseptual yang jelas.